

From Human-Centric to Machine-Optimized: A Technolinguistic Analysis of Documentation Readability in `llms.txt` Standards

Yehezkiel Gunawan

Information Systems Department
BINUS Online Learning
Bina Nusantara University
Jakarta, Indonesia
yehezkiel.gunawan@binus.ac.id

Tanty Oktavia

Information Systems Management Department
BINUS Graduate Program
Bina Nusantara University
Jakarta, Indonesia
toktavia@binus.edu

Hadi Syahrial

Information Systems Management Department
BINUS Graduate Program
Bina Nusantara University
Jakarta, Indonesia
hadi.syahrial@binus.ac.id

This study compares human-readable HTML documentation and machine-oriented `llms.txt` files across ten software engineering platforms. Flesch Reading Ease (FRE), Flesch-Kincaid Grade Level, lexical density, and token-to-word ratio are combined with an LLM-as-a-Judge evaluation and a preliminary LLM Readability Index (LRI). Machine documentation had a 91.58-point lower mean FRE but a 14.69-point higher mean LRI ($p = 0.005$, $d = -1.172$). Clarity, completeness, conciseness, and LLM-friendliness favored machine documentation with large effects; Technical Accuracy did not differ significantly. No statistically significant correlation was detected between traditional readability metrics and LLM ratings. These findings identify a readability-tokenization paradox in measurement outcomes: formats rated favorably by the selected evaluator can score poorly on human-oriented readability formulas. LRI is an operational composite, not a validated readability instrument, and the study does not establish improved downstream LLM performance.

Keywords—technolinguistics, documentation readability, LLM-as-a-Judge, `llms.txt`, Flesch Reading Ease, LLM Readability Index, technical documentation, tokenization

I. INTRODUCTION

LLM-based assistants increasingly mediate access to software engineering documentation, creating a second audience alongside human developers. Human-oriented documentation is typically organized and evaluated for comprehension, whereas machine-oriented formats favor compact, structured access within finite context windows. This raises a measurement question: do formats designed for machine consumption trade traditional readability for qualities that an LLM evaluator rates more favorably?

Readability has long guided technical communication. FRE and FKGL operationalize difficulty through sentence length and syllable counts and remain embedded in widely used writing tools. Those formulas, however, were calibrated for human readers and continuous prose. They do not account for navigation, retrieval structure, code, Markdown syntax, or the needs of an LLM consuming documentation as context. Applying them unchanged to machine-oriented files may therefore conflate linguistic difficulty with structural difference.

Jeremy Howard introduced the `llms.txt` standard in September 2024 as a compact, Markdown-based source for inference-time use. Its emergence parallels earlier changes from print to HTML, but established readability formulas were designed for continuous human prose rather than link directories, API listings, or token-oriented structures. Byte Pair

Encoding further complicates evaluation because vocabulary and markup affect token cost independently of sentence-level readability. The “Lost in the Middle” effect also motivates concise, front-loaded access to documentation [1].

Developer-facing AI assistants now retrieve documentation to answer questions, explain APIs, and support coding tasks. This use makes documentation quality consequential, but it does not follow that a file labeled for LLM use improves those tasks. A measurement design must separate traditional readability, tokenization behavior, evaluator preference, and downstream performance. The present study addresses the first three and treats the fourth as an external-validation requirement.

This study addresses the gap through a quantitative, platform-matched paired comparison of human-readable HTML and machine-oriented documentation from ten software engineering platforms. The sample spans frameworks, developer platforms, infrastructure, payments, AI tools, and backend services. Traditional readability and tokenization measures are compared with five LLM-as-a-Judge dimensions and a preliminary composite LRI. Platform matching supports within-platform comparison but does not imply that paired corpora are content-identical.

The contribution is threefold: empirical comparison of human- and machine-oriented documentation, measurement of the divergence between traditional metrics and LLM ratings, and a reproducible framework for examining this divergence. The resulting readability-tokenization paradox concerns measurement outcomes rather than demonstrated gains in retrieval, generation, or user performance.

A. Research Questions

This study addresses three primary research questions:

RQ1: Do machine-oriented documentation files (`llms.txt`) have markedly different traditional readability scores than their human-readable HTML counterparts?

RQ2: Do the LLM-as-a-Judge evaluation scores favor machine-oriented documentation over human-readable HTML documentation?

RQ3: Are existing readability measures (Flesch Reading Ease, Flesch-Kincaid Grade Level) sufficient for assessing documentation meant for both human and machine consumption, or do they fail to reflect LLM-perceived quality dimensions?

B. Significance of the Study

This study contributes to technolinguistics by quantifying how human-oriented readability formulas and LLM-based quality ratings diverge. It offers documentation teams a basis

for complementary evaluation while positioning LRI as a preliminary composite requiring external validation.

II. LITERATURE REVIEW

A. The *llms.txt* Standard and Machine-Oriented Documentation

Howard proposed *llms.txt* to provide LLMs with concise, structured website information at inference time [2]. The Markdown format uses a title, summary, curated links, and an optional section whose content can be omitted under tighter context constraints. It therefore separates machine access from HTML navigation, advertising, and client-side presentation. Vercel, Cloudflare, Postman, Cursor, Anthropic, and Hugging Face are among its adopters [3]; related tools generate the format for major documentation systems, while variants such as *llms-full.txt* aggregate fuller content [4], [5]. This ecosystem makes explicit the tension examined here: a document can be concise and structured for machine access without satisfying conventions developed for human prose.

The proposal responds to a practical retrieval problem: converting complex websites with navigation and JavaScript into plain text is noisy, while placing an entire site in context is often infeasible. A curated root file can expose summaries and links, and a fuller variant can aggregate source material for systems with larger budgets. Implementations nevertheless vary substantially. Some sites provide short indexes, others publish prose-rich topic collections, and others expose very large aggregates. Thus, *llms.txt* denotes an access convention rather than a uniform textual genre.

B. Traditional Readability Metrics

The Flesch Reading Ease (FRE) formula, developed by Rudolf Flesch in 1948 [6], and the Flesch-Kincaid Grade Level (FKGL), developed by J. Peter Kincaid et al. in 1975 [7], are the foundation of modern readability measurement. The FRE formula uses sentence length and syllable count per word to compute readability; FKGL converts these metrics into U.S. grade level equivalents.

These formulas have gained academic credibility. Flesch’s original research appeared in the *Journal of Applied Psychology*, and FKGL became a U.S. military standard in 1978. The U.S. Department of Defense, Microsoft Word, Grammarly, and IBM Lotus Symphony have all adopted these measures. Yet Redish (2000) [8] and McClure (1987) [9] point out serious limitations when applying these formulas to technical documentation: they were built for schoolbooks, not technical content; they ignore reader differences and the effects of content, layout, and retrieval aids; and they assume English text, limiting their applicability to non-English contexts.

Industry standards for technical documentation suggest grade levels of 8-10 for general developer audiences and FRE scores of 60-70 for general documentation [10]. Technical content often scores lower because readability formulas penalize polysyllabic terminology. This limitation matters for machine-oriented documentation, which may use dense, concise language that traditional metrics rate as less readable.

Alternative formulas such as ARI, Coleman-Liau, SMOG, and Gunning Fog vary their treatment of vocabulary and sentence complexity but remain surface-based and human-centered. Coh-Metrix adds cohesion and semantic overlap at greater computational cost. Neural models can capture context, yet models trained on human judgments retain a human-oriented target. Applying any of these measures to concise instructions, link directories, and API listings is therefore

uncertain. This study treats LRI as complementary to, rather than equivalent to, human readability measures.

This construct distinction is important in technical domains. Long identifiers and polysyllabic product terms can depress formula scores even when they are familiar to the intended audience. Conversely, a short link label can appear linguistically simple while providing insufficient context. Layout, prior knowledge, examples, and retrieval aids also shape human comprehension but are absent from FRE and FKGL. The metrics remain useful as standardized surface indicators, provided their outputs are not interpreted as direct comprehension measurements for non-prose artifacts.

C. Tokenization and Computational Efficiency

Byte Pair Encoding (BPE), introduced to NLP by Sennrich, Haddow, and Birch in 2016 [11], is the standard tokenization method for modern LLMs. BPE is lossless, reversible, and compresses text, typically producing tokens of about 4 bytes each [12]. Variable-length encoding is its key feature: common words need only one token, while rare or complex words need several. Recent work has examined how tokenization choices affect model behavior [13], and the selection of subword merge operations has a notable effect on downstream performance [14].

Token count affects API cost, latency, and context-window use. The “Lost in the Middle” phenomenon, in which models use information less reliably when it appears within long contexts, motivates concise and front-loaded formats such as *llms.txt* [1].

Token efficiency is not equivalent to brevity in words. Rare technical terms, URL fragments, punctuation, and Markdown can split into multiple tokens, so a compact directory may have a high TWR even when it contains little prose. TWR is included to expose this computational property without treating it as a direct measure of model comprehension or task success.

LLMLingua (Jiang et al., 2023), published at EMNLP 2023, demonstrates that well-structured compression can preserve semantic integrity while reducing token count [15], achieving up to 20x prompt compression with minimal performance loss. This work, however, focuses on compression techniques rather than documentation standards. It leaves a gap in the understanding of how documentation format choices affect machine efficiency and human readability.

D. LLM-as-a-Judge Evaluation Methodology

LLM-as-a-Judge methods assess semantic qualities that formula-based metrics omit, including clarity, completeness, conciseness, and technical accuracy. Strong judges can exceed 80% agreement with human preferences in some settings [16], and related work has applied LLM evaluation to code readability [17], [18] and generated documentation [19]. These results support scalable evaluation but do not remove model-specific bias or establish downstream utility.

Judge outputs depend on the selected model, rubric, prompt, scale, and aggregation procedure. Position bias, score compression, and self-preference have been documented in adjacent evaluation settings. For this reason, the five dimensions in this study are reported individually as well as through LRI, and Technical Accuracy is not assumed to behave like presentation-oriented dimensions.

E. Research Gap

Despite the extensive literature on readability metrics, tokenization efficiency, and LLM evaluation, a key gap remains: no peer-reviewed study has systematically compared

TABLE I: Platform Characteristics

Platform	Category	Domain
React	UI Framework	Frontend
GitHub	Developer Platform	DevOps
Supabase	Backend-as-a-Service	Backend
Vercel	Deployment Platform	DevOps
Stripe	Payment Platform	Fintech
LangChain	AI Orchestration	AI/ML
Cloudflare	Infrastructure	DevOps
Cursor	AI Editor	Developer Tools
Hono	Web Framework	Backend
FastAPI	Python Framework	Backend

traditional readability metrics with LLM evaluation scores for machine-oriented documentation. This study addresses that gap across ten technical documentation platforms.

III. METHOD

A. Research Design

This study uses quantitative content analysis and a platform-matched paired design. Human-readable HTML documentation and machine-oriented files from the same platforms are compared using traditional computational-linguistics metrics and an LLM-as-a-Judge evaluation. Platform matching reduces domain variation but does not make the paired corpora content-identical.

The platform is the unit of pairing. Scores were aggregated within each platform and documentation type before paired inference, avoiding treatment of chunks as independent platform observations. The design supports comparison of formats as they existed in practice; it does not isolate format from differences in coverage, implementation maturity, or source composition.

B. Corpus Selection

The corpus includes ten software engineering documentation platforms selected for their adoption of `llms.txt`-style documentation and representation of diverse technical domains. Selection required (1) human-readable HTML and machine-oriented documentation, (2) a technical rather than marketing focus, (3) English content, and (4) public accessibility.

Purposive selection provided variation across frontend and backend frameworks, developer and deployment platforms, infrastructure, payments, AI orchestration, and backend services. This diversity reduces dependence on a single software domain, but it does not support population-level estimates for all technical documentation. Table I describes the sample.

C. Data Collection

Data were collected with a custom Python CLI that separates acquisition, cleaning, measurement, evaluation, and export. For the human corpus, Crawl4AI and Playwright performed breadth-first crawling to a maximum depth of three and 50 pages per platform, including JavaScript-rendered sites. Mozilla Readability extracted prose into Markdown; navigation, code blocks, images, advertisements, and scripts were removed while headings, paragraphs, and link anchors were retained [20]. Cleaned outputs below 500 characters were flagged for alternative extraction or manual review.

The crawler restricted traversal to documentation-relevant pages and converted rendered HTML to Markdown before prose extraction. Removing code prevents source listings from dominating sentence and lexical calculations, but this choice also means that the human corpus represents explanatory prose rather than every element visible on the original page. Headings and anchors were retained because they contribute to how readers identify and retrieve topics.

The machine corpus was assembled by checking each root domain for `/llms.txt` and the legacy `/llm.txt` endpoint, then following linked text files in their listed order. Expanded variants such as `llms-full.txt` and `llms-ctx.txt` were included when available. Context7 supplied fallback content when no direct file existed or when it returned a more complete source. Human files averaged approximately 150,000 characters per platform, whereas machine files ranged from 3,000 to 8.8 million characters, reflecting variation from concise summaries to full-documentation aggregates. Appendix A shows representative excerpts.

Relative links were resolved against each platform domain, and linked text files were concatenated in source order. When Context7 was considered, content length determined whether its version replaced the direct source. This rule favored coverage but did not guarantee topical equivalence with the crawled human pages. Source files were retained so that corpus construction and the structural heterogeneity visible in Appendix A could be inspected.

Long documents were divided into approximately 4,000-token chunks. The model `nvidia/nemotron-nano-12b-v2-v1` evaluated each chunk through OpenRouter [21] using a structured prompt and JSON response schema; dimension scores were averaged equally across chunks. Cached responses supported resumable processing. Figure 1 summarizes the collection and analysis flow.

The prompt supplied definitions and a 1–5 rubric for every dimension and required parseable scores. Temperature was set to 0.0 to reduce stochastic variation. Each chunk contributed equal weight to the platform score, including a shorter final chunk; this choice simplified aggregation but is treated as a limitation. The model was selected to keep the procedure accessible and reproducible rather than to claim evaluator consensus.

D. Metrics and Instruments

1) Traditional Readability Metrics

The study uses three traditional readability metrics implemented with the Python `textstat` library:

Flesch Reading Ease (FRE): Calculated as $206.835 - 1.015(\text{total words}/\text{total sentences}) - 84.6(\text{total syllables}/\text{total words})$. Scores range from 0 to 100, with higher scores indicating easier readability.

Flesch-Kincaid Grade Level (FKGL): Calculated as $0.39(\text{total words}/\text{total sentences}) + 11.8(\text{total syllables}/\text{total words}) - 15.59$. Scores represent U.S. grade level equivalents.

Lexical Density (LD): Calculated as the percentage of content words (nouns, verbs, adjectives, adverbs) relative to total words, using the NLTK stopwords corpus for stopword identification.

Token-to-Word Ratio (TWR): Calculated using OpenAI’s `tiktoken` library with the `cl100k_base` encoding (GPT-4). This metric measures the ratio of tokens to words, indicating tokenization efficiency.

2) LLM-as-a-Judge Evaluation

The LLM evaluation uses a five-dimension Likert scale assessment via the OpenRouter API [21], using the `nvidia/nemotron-nano-12b-v2-v1` model. The evaluation dimensions are:

- 1) **Clarity** (1-5): Is the documentation clear and unambiguous?
- 2) **Completeness** (1-5): Does it cover the topic adequately?
- 3) **Conciseness** (1-5): Is it free of unnecessary verbosity?

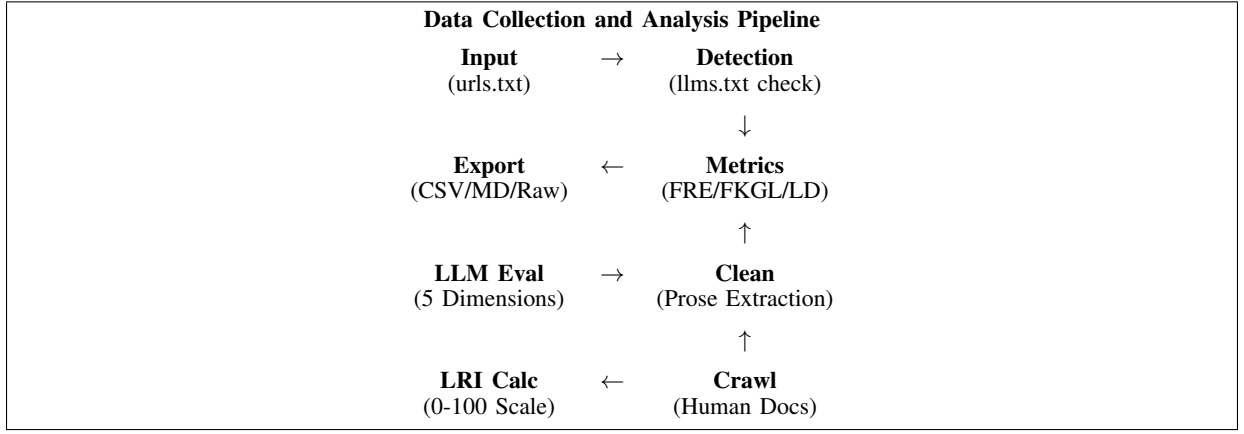


FIG. 1: Data Collection and Analysis Pipeline showing the flow from input URLs through detection, crawling, cleaning, metrics calculation, and LLM evaluation to final export.

- 4) **Technical Accuracy** (1-5): Are technical details correct?
- 5) **LLM-Friendliness** (1-5): Is it optimized for machine consumption?

LLM Readability Index (LRI): The LRI is proposed as a preliminary operational composite of LLM-perceived documentation quality. Let s_{ij} denote the score assigned to document i on dimension j , where $s_{ij} \in [1, 5]$. The equally weighted mean score is:

$$\bar{s}_i = \frac{1}{5} \sum_{j=1}^5 s_{ij} \quad (1)$$

The mean is then linearly normalized from the 1–5 range to a 0–100 range:

$$\text{LRI}_i = 100 \left(\frac{\bar{s}_i - 1}{5 - 1} \right) = 25(\bar{s}_i - 1) \quad (2)$$

Equal weights are assigned because no validated empirical basis currently exists for differential weighting of the five dimensions. The transformation yields 0 when all dimension scores are 1, 50 when their mean is 3, and 100 when all scores are 5. It preserves rankings and inferential conclusions while providing a common numerical range for presentation. Sharing a 0–100 range with Flesch Reading Ease does not imply that the two measures represent equivalent constructs; LRI reflects the evaluator’s composite assessment, whereas Flesch Reading Ease reflects surface-level linguistic complexity.

E. Analysis Procedure

1) Statistical Tests

The analysis uses paired samples t-tests to compare human-readable and machine-oriented documentation for the same platforms. Effect sizes are calculated using Cohen’s d for paired samples:

$$d = \frac{\text{mean difference}}{\text{standard deviation of differences}} \quad (3)$$

Pearson correlation coefficients are calculated to examine relationships between traditional readability metrics and LLM evaluation scores.

All tests used two-sided $\alpha = 0.05$. Means and standard deviations summarize format-level distributions, and medians and trimmed analyses assess sensitivity to extreme FRE and TWR observations. Statistical significance and effect magnitude are interpreted separately, particularly for Technical Accuracy and the correlation analysis.

2) Software and Tools

Python 3.11+, textstat, tiktoken (cl100k_base), NLTK, SciPy, pandas, NumPy, Matplotlib, and seaborn supported measurement, analysis, and visualization. A standalone analysis script reads exported CSV data and regenerates the statistical tables and reports.

3) Validation Steps

Validity checks covered five areas. First, random samples were compared across raw pages, cleaned Markdown, and metric inputs to assess extraction quality. Second, manual FRE and FKGL calculations for FastAPI, React, and Stripe differed by less than 0.5% from automated results. Third, robust statistics and trimmed means assessed the influence of React (FRE = −430.09) and LangChain (−234.73); removing them changed mean machine FRE from −58.18 to 12.45 without reversing the format-level direction. Fourth, cached outputs preserve the evaluated scores and provenance, although fresh API calls may vary. Finally, re-evaluation of five document pairs one week later produced $r = 0.94$, indicating high temporal consistency for the selected judge. Automated tests additionally covered acquisition, cleaning, measurement, evaluation, and export behavior.

Manual inspection focused on whether navigation was removed without discarding the principal explanatory content. Metric checks applied the published formulas to 500-word samples before comparison with automated outputs. The sensitivity analysis is especially important because negative FRE values are mathematically possible but become difficult to interpret when sentence boundaries are sparse. The temporal consistency check measures repeatability for this evaluator and sample; it is not inter-model agreement or external construct validation.

F. AI Declaration

AI was used for the LLM-as-a-Judge evaluation following established protocols [16], for assistance with analysis code and visualization, and for literature retrieval. The primary author retained control of research design, interpretation, and conclusions.

IV. RESULT AND DISCUSSION

A. Traditional Readability Analysis

The ten platform pairs span frameworks, services, developer tools, and AI platforms. Figure 2 contrasts FRE and LRI: human documentation generally has higher FRE, whereas machine documentation generally has higher LRI. React and

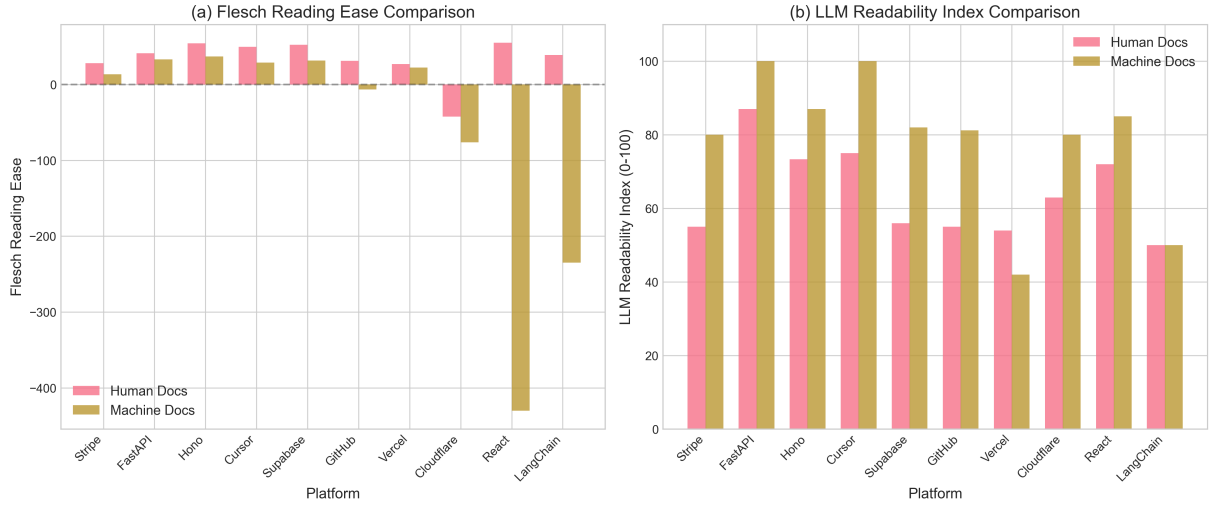


FIG. 2: Platform-level comparison of (a) Flesch Reading Ease and (b) LLM Readability Index scores. Human documentation (pink) generally has higher FRE, while machine documentation (gold) generally has higher LRI. React and LangChain illustrate the extreme variance in machine FRE.

TABLE II: Traditional Readability Metrics Comparison

Metric	Human	Machine	Cohen's d	Effect
Flesch Reading Ease	33.39 (28.66)	-58.18 (155.28)	0.573	Medium
Flesch-Kincaid Grade	13.76 (6.18)	23.96 (20.91)	-0.459	Small
Lexical Density (%)	76.86 (9.23)	77.25 (8.70)	-0.028	Negligible
Token-to-Word Ratio	1.67 (0.36)	2.73 (1.48)	-0.661	Medium

Values are mean (SD). Cohen's d calculated for paired samples. Negative d indicates machine docs score higher than human docs.

LangChain are extreme machine-FRE outliers.

Table II shows mixed traditional-metric outcomes. Human documentation has a 91.58-point higher mean FRE, but machine scores vary widely ($SD = 155.28$), driven partly by link-heavy React (-430.09) and LangChain (-234.73) files for which prose-based formulas have weak construct validity. Machine documentation also has a 10.2-level higher mean FKGL. Lexical density is nearly equal across formats, while mean TWR is higher for machine documentation (2.73 versus 1.67), contrary to a simple token-efficiency expectation. React and LangChain exceed 5.0 TWR, showing that implementation structure strongly affects tokenization.

The direction of FRE favors human documentation with a medium paired effect ($d = 0.573$), whereas the FKGL effect is smaller ($d = -0.459$). Because the inferential tests for these traditional metrics are not reported, these differences are interpreted descriptively rather than as statistically significant format effects. The near-zero lexical-density effect ($d = -0.028$) indicates that the formats contain similar proportions of content-bearing words even though their sentence structure and markup differ.

The TWR pattern also qualifies the premise that machine-oriented files are inherently token-efficient. A ratio counts token fragments per whitespace-delimited word, not total context length or useful information per token. URLs, punctuation, identifiers, and Markdown links can raise the ratio, while a longer prose document may still contain more tokens overall. TWR therefore identifies one aspect of computational representation rather than an overall efficiency score.

B. LLM-as-a-Judge Evaluation Results

The selected evaluator rated machine documentation higher on LRI ($M=78.72$) than human documentation ($M=64.03$), a mean difference of 14.69 points ($t = -3.706$, $p = 0.005$, $d = -1.172$, 95% CI [4.89, 24.49]). Conciseness ($p = 0.006$), LLM-Friendliness ($p = 0.007$), Completeness ($p = 0.014$), and Clarity ($p = 0.015$) also favored machine documentation with large effects. Technical Accuracy showed only a small,

non-significant difference ($d = -0.245$, $p = 0.455$). Table III reports the dimension-level results, and Figure 3 summarizes the effect sizes without implying downstream performance.

The dimension pattern localizes the aggregate difference. Conciseness ($d = -1.347$) and LLM-Friendliness ($d = -1.371$) show the largest effects, followed by Clarity ($d = -1.002$) and Completeness ($d = -0.964$). These are presentation- and access-oriented criteria that the `llms.txt` convention explicitly targets. Technical Accuracy remains high in both formats (4.20 human and 4.36 machine), and its non-significant difference provides no evidence that either format is more correct.

Eight of ten platform pairs favor machine documentation. Vercel is the only exception (42.00 machine versus 54.00 human), while LangChain is tied at 50.00. FastAPI and Cursor reach the scale ceiling of 100.00, suggesting potential score compression. Positive deltas range from 13.00 for React and FastAPI to 26.25 for GitHub. This spread, together with Vercel's negative delta, indicates substantial platform-level heterogeneity and shows that format labels alone do not determine evaluator ratings.

C. Correlation Analysis

No statistically significant correlation was detected between traditional metrics and LLM ratings (all $p > 0.05$). FRE and LLM-Friendliness showed $r = 0.357$ ($p = 0.312$); TWR and LLM-Friendliness showed the largest observed coefficient, $r = 0.564$ ($p = 0.090$), which fell to $r = 0.312$ ($p = 0.412$) after removing the React and LangChain TWR outliers. Low-FRE machine files could still receive high Clarity ratings, illustrating divergence between the measures, but the small sample does not establish independence between constructs.

D. The Readability-Tokenization Paradox

The readability-tokenization paradox describes the observed divergence between human-oriented formulas and LLM-based ratings: machine documentation generally received lower FRE and higher LRI. This is not proof that

TABLE III: LLM-as-a-Judge Evaluation Results

Dimension	Human	Machine	Cohen's d	Effect
Clarity	3.50 (0.51)	4.08 (0.71)	-1.002	Large
Completeness	3.38 (0.51)	4.01 (0.78)	-0.964	Large
Conciseness	3.38 (0.58)	4.14 (0.81)	-1.347	Large
Technical Accuracy	4.20 (0.59)	4.36 (0.89)	-0.245	Small
LLM-Friendliness	3.35 (0.58)	4.16 (0.82)	-1.371	Large
LRI (Overall)	64.03 (12.13)	78.72 (18.88)	-1.172	Large

Values are mean (SD) on 1-5 Likert scale. LRI = LLM Readability Index (0-100). Negative Cohen's d indicates machine docs score higher.

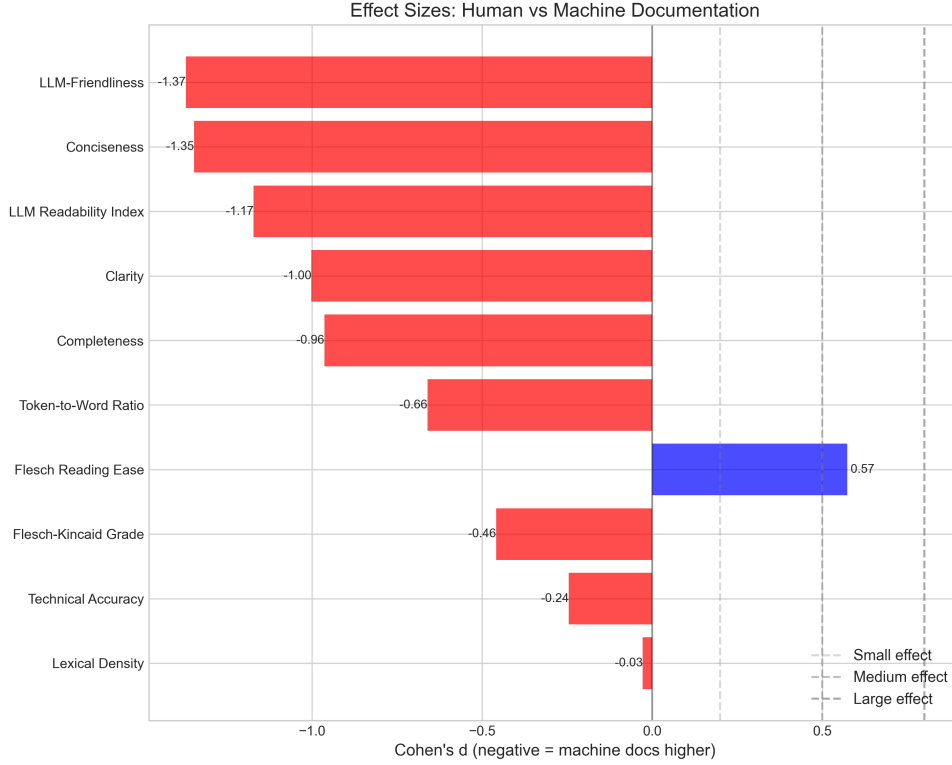


FIG. 3: Cohen's d effect sizes for all metrics and evaluation dimensions. Negative values favor machine documentation and positive values favor human documentation. Clarity, Completeness, Conciseness, LLM-Friendliness, and LRI show large machine-favoring effects; Technical Accuracy shows a small effect.

machines process difficult text more effectively. FRE and FKGL respond to sentence length and syllables, whereas the judge considered broader documentation qualities; TWR separately captures tokenizer fragmentation and markup overhead. Complementary measures are therefore needed for dual-audience documentation.

The term “paradox” denotes competing evaluation outcomes, not a logical contradiction. Machine-oriented files can contain dense labels, URLs, and fragments that violate assumptions of prose formulas while presenting a compact hierarchy that the judge rewards for clarity or conciseness. Human files can provide explanation, narrative transitions, and examples that support comprehension while receiving lower conciseness ratings. Neither outcome alone establishes which document better supports a reader or application.

This distinction also explains why higher mean TWR does not negate higher LRI. LRI includes the evaluator's judgments of organization and coverage, whereas TWR reflects lexical segmentation. A file can be fragmented at the token level yet still expose topics in a structure the evaluator rates favorably. The results therefore identify tension among operationalizations rather than a single continuum from human-readable to machine-readable.

E. Boundary Conditions and Methodological Interpretation

The pattern is neither universal nor format-deterministic. Vercel's machine file received a lower LRI than its human

counterpart, and LangChain showed no difference. React and LangChain also demonstrate that applying prose formulas to link directories or API listings can produce artifacts rather than interpretable reading difficulty. Moreover, none of the tested correlations reached statistical significance, but $n = 10$ provides limited power; the results do not establish that the measures are independent.

Vercel is particularly informative because it prevents the format label from functioning as an explanation by itself. Its machine source is unusually link-heavy and received lower evaluator ratings for the qualities represented in LRI. LangChain's tie likewise shows that a separate machine artifact need not yield an incremental rating. These cases suggest that source coverage, explanatory context, and implementation choices may matter as much as the presence of a standard endpoint.

React and LangChain define another boundary: readability formulas require enough sentence-like prose for averages of words per sentence and syllables per word to be meaningful. Directory entries and API signatures can generate extreme scores because punctuation and segmentation are interpreted as prose structure. The outlier-sensitive FRE mean should consequently be read alongside the robust analysis, sample excerpts, and platform-level values rather than as a literal estimate of human comprehension.

The LLM findings likewise concern one evaluator's ratings.

The study did not test retrieval accuracy, generated-answer quality, code generation, latency, cost, or user performance. LRI combines five equally weighted dimensions and equally weighted chunks on a convenient 0–100 scale. It remains a preliminary operational composite of evaluator-perceived documentation quality, not a validated readability instrument.

The non-significant correlations require similar restraint. Failure to reject zero association is not evidence that an association is absent, especially with ten paired platforms and influential outliers. The results show that the observed ratings were not well summarized by FRE or FKGL in this corpus. Replication with greater power is needed before making stronger claims about construct relationships.

Equal weighting makes LRI transparent and reproducible, but transparency is not validation. Completeness and Technical Accuracy are broader documentation-quality constructs than readability, and a short final chunk contributes as much as a full chunk. Alternative weights, factor analysis, token-weighted aggregation, and criterion validation may produce different composites. Reporting the dimensions separately remains essential.

F. Practical and Technolinguistic Implications

Documentation teams may use human readability measures and machine-oriented evaluation as complementary diagnostics rather than optimize for either score alone. Maintaining human HTML alongside carefully curated `llms.txt` content is a plausible strategy, but Vercel shows that providing a machine-oriented file does not guarantee a favorable rating. The findings contribute to technolinguistics by locating the tension in evaluation criteria: human comprehension, tokenization, and LLM-assessed documentation quality overlap but are not interchangeable. External validation is required before these results can support prescriptive claims about downstream systems.

In practice, teams can audit human prose for sentence-level accessibility while separately checking machine artifacts for coverage, hierarchy, link quality, and token characteristics. A dual-format strategy avoids forcing one artifact to satisfy incompatible surface conventions, but it also creates maintenance and synchronization costs not measured here. A well-structured shared source that generates both outputs may be preferable where content parity can be verified.

Theoretically, the findings extend technolinguistic analysis from language addressed directly to humans toward language selected, structured, or transformed for computational intermediaries. The relevant unit is not only the sentence but also the interface through which content enters a model context. This perspective preserves established readability research while recognizing that machine-mediated communication requires additional constructs and external task-based evidence.

V. CONCLUSION

This platform-matched study quantifies a readability-tokenization paradox in technical documentation. For RQ1, machine-oriented files generally had lower traditional readability scores, although extreme non-prose files made FRE highly sensitive to outliers; mean FKGL was 10.2 levels higher. For RQ2, the selected evaluator rated machine documentation 14.69 LRI points higher on average ($p = 0.005$, $d = -1.172$), with large effects for Clarity, Completeness, Conciseness, and LLM-Friendliness but not Technical Accuracy. For RQ3, no statistically significant correlation was detected between traditional metrics and LLM ratings, so human-oriented formulas alone do not represent the broader qualities included

in this evaluation.

The contribution is a reproducible comparison of these measurement perspectives and a transparent, preliminary LRI composite. For documentation practice, the results support complementary human- and machine-oriented assessment and motivate, but do not mandate, dual-format publication. LRI requires external validation, and the study did not measure retrieval, question answering, code generation, or human task performance.

Accordingly, the evidence supports a scoped conclusion: machine-oriented documentation was usually rated more favorably by this judge despite performing poorly on formulas designed for human prose. Whether that rating predicts better technical outcomes remains an open empirical question.

A. Limitations

Corpus and construct comparability. Platform matching does not ensure content equivalence, and implementations range from prose-rich documents to link directories. FRE and FKGL were designed for continuous prose, so React’s link-heavy file, for example, yields an extreme FRE (−430.09) that mixes linguistic complexity with structural artifacts. Format-level comparisons therefore also reflect implementation heterogeneity.

Human sources were crawled and cleaned, whereas machine sources were downloaded or aggregated through their published links and, where necessary, Context7. Differences can therefore arise from coverage, extraction, and source selection as well as format. Code was removed from human pages, while structured elements remained prominent in some machine files. A content-controlled experiment would be required to isolate formatting alone.

LLM-judge and LRI validity. One 12B evaluator was used at temperature 0.0. Provider and numerical variation remain possible, while LangChain’s neutral scores and the FastAPI and Cursor ceilings suggest limited score resolution. LRI combines readability-related and broader quality dimensions with equal dimension and chunk weights. Its 0–100 transformation establishes neither equivalence with FRE nor predictive validity for downstream tasks [22].

The temporal repeatability result does not establish agreement with human experts or other models. The evaluator may also favor conventions resembling its training data or the language of the rubric. Because no retrieval, QA, or generation benchmark was used, high LRI should be interpreted as favorable rubric-based assessment only. Multi-model and human calibration are needed before selecting thresholds or differential weights.

Sampling and generalizability. The purposive sample contains ten English-language platforms, limiting population and cross-linguistic inference. With $n = 10$, the minimum detectable Pearson correlation is approximately $r = 0.63$ at $\alpha = 0.05$; non-significant correlations do not demonstrate independence. The observational design also precludes causal claims about formatting.

The sampled domains are diverse but omit hardware, enterprise software, scientific libraries, and non-English documentation. Flesch measures are specifically calibrated to English syllable patterns, while tokenizer behavior also varies across languages and scripts. Results should not be generalized to these settings without replication.

Temporal and pipeline validity. The corpus captures one early-2025 window in an evolving standard. Crawling, JavaScript rendering, cleaning, and the 500-character threshold can introduce omissions or selection artifacts. Longitudinal

replication is needed to separate format patterns from documentation drift.

Published documentation and machine endpoints can change independently, so later collections may not reproduce the same content even when the code is unchanged. Cached judge responses preserve the analyzed observations but do not resolve source drift or fresh-inference variation. Versioned snapshots and repeated collection windows would strengthen temporal validity.

B. Future Work

Future work should use larger and multilingual corpora, multiple evaluator models, and content-controlled format comparisons. Human comprehension studies and downstream retrieval, question-answering, and code-generation experiments are needed to test the external validity of LRI and the practical consequences of machine-oriented documentation. Longitudinal sampling could track whether implementations converge as the standard matures, while factor and weighting studies could determine whether the five LRI dimensions form a stable composite.

ACKNOWLEDGMENT

The author acknowledges the use of AI tools for statistical analysis, visualization, and literature review synthesis. The LLM-as-a-Judge evaluation was conducted using the nvidia/nemotron-nano-12b-v2-v1 model via the OpenRouter API. The author maintains full responsibility for research design, data interpretation, and conclusion formulation.

AUTHOR CONTRIBUTIONS

Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, visualization, and writing—original draft, Yehezkiel Gunawan; supervision and project administration, Tanty Oktavia and Hadi Syahrial. All authors reviewed and approved the final manuscript.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available in the GitHub repository at <https://github.com/yehezkielgunawan/codhes-2026/tree/main/readability-python-demo>.

APPENDIX: REPRESENTATIVE DOCUMENTATION SAMPLES

The following excerpts illustrate the structural contrast between cleaned human-readable documentation and the retrieved machine-oriented files. Ellipses mark omissions; complete samples are available in the repository.

A.1 FastAPI

Human-readable documentation excerpt:

```
# FastAPI
FastAPI framework, high performance, easy to learn,
fast to code, ready for production.
```

```
FastAPI is a modern, fast (high-performance) web
framework for building APIs with Python based on
standard Python type hints.
```

```
The key features are: Fast, Fast to code, Fewer bugs,
Intuitive, Easy, Short, Robust, and Standards-based.
...
```

Machine-oriented documentation excerpt:

```
### Add Summary and Description to FastAPI Path Operations
Source: github.com/fastapi/fastapi/...
```

```
Provide a concise summary and a more detailed description
for a path operation, improving its clarity in OpenAPI.
```

```
'''python
@app.post("/items/{name}", summary="Create an item")
async def create_item(name: str):
    return {"name": name}
'''
...
```

A.2 React

Human-readable documentation excerpt:

```
### GET STARTED
* Quick Start
* Tutorial: Tic-Tac-Toe
* Thinking in React
* Installation
* Creating a React App
* Build a React App from Scratch
* Setup
* Editor Setup
* Using TypeScript
...
```

Machine-oriented documentation excerpt:

```
# React Documentation
> The library for web and native user interfaces.
```

```
## Learn React
### GET STARTED
#### Quick Start
- [Quick Start](https://react.dev/learn.md)
- [Tutorial: Tic-Tac-Toe](...)
- [Thinking in React](...)
...
```

REFERENCES

- [1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 11, 2023.
- [2] J. Howard. (2024, September 3) The /llms.txt file: A proposal to standardise on using an /llms.txt file to provide information to help LLMs use a website at inference time. [Online]. Available: <https://llmstxt.org/>
- [3] AnswerDotAI. (2024) llms-txt. GitHub. [Online]. Available: <https://github.com/AnswerDotAI/llms-txt>
- [4] (2024) llmstxt.site. [Online]. Available: <https://llmstxt.site/>
- [5] (2024) directory.llmstxt.cloud. [Online]. Available: <https://directory.llmstxt.cloud/>
- [6] R. Flesch, "A new readability yardstick," *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948.
- [7] J. P. Kincaid, R. P. Fishburne, R. L. Rogers, and B. S. Chissom, "Derivation of new readability formulas (automated readability index, fog count and flesch reading ease formula) for navy enlisted personnel," Naval Air Station Memphis, Research Branch Report 8-75, 1975. [Online]. Available: <https://apps.dtic.mil/sti/pdfs/ADA006655.pdf>
- [8] J. C. Redish, "Readability formulas have even more limitations than Klare discusses," *ACM Journal of Computer Documentation*, vol. 24, no. 3, pp. 132–137, 2000.
- [9] G. McClure, "Readability formulas: Useful or useless? (an interview with J. Peter Kincaid)," *IEEE Transactions on Professional Communication*, vol. 30, no. 1, pp. 12–15, 1987.
- [10] J. P. Kincaid, R. Braby, and J. Mears, "Electronic authoring and delivery of technical information," *Journal of Instructional Development*, vol. 11, no. 2, pp. 8–13, 1988.
- [11] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)*, Berlin, Germany, 2016, pp. 1715–1725.
- [12] OpenAI. (2023) tiktoken: A fast BPE tokeniser for use with OpenAI's models. GitHub. [Online]. Available: <https://github.com/openai/tiktoken>
- [13] Z. Ankner, A. Bertsch, G. Neubig, and M. R. Gormley, "The "·" tokenization," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [14] S. Ding, A. Renduchintala, and K. Duh, "A call for prudent choice of subword merge operations in neural machine translation," in *Proceedings of MT Summit 2019*, 2019. [Online]. Available: <https://aclanthology.org/W19-6608/>
- [15] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 2023, pp. 18 020–18 036.
- [16] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Proceedings of the Neural Information Processing Systems (NeurIPS) 2023 Datasets and Benchmarks Track*, 2023.
- [17] K. Horikawa, K. Horikawa, Y. Kashiwa, H. Uwano, and H. Iida, "Do AI agents really improve code readability?" in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.13723>
- [18] I. R. d. S. Simoes and E. Venson, "Measuring how changes in code readability attributes affect code quality evaluation by large language models," in *Proceedings of the XXXIX Brazilian Symposium on Software Engineering*, 2025.

- [19] C. Diggs, M. Doyle, A. Madan, S. Scott, E. Escamilla, J. Zimmer, N. Nekoo, P. Ursino, M. Bartholf, Z. Robin, A. Patel, C. Glasz, W. Macke, P. Kirk, J. Phillips, A. Sridharan, D. Wendt, S. Rosen, N. Naik, J. F. Brunelle, and S. Thaker, "Leveraging LLMs for legacy code modernization: Challenges and opportunities for LLM-Generated documentation," in *Proceedings of the LLM4Code Workshop at ICSE 2025*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.14971>
- [20] Mozilla. (2024) Readability: A standalone version of the readability library used for Firefox reader view. Mozilla. [Online]. Available: <https://github.com/mozilla/readability>
- [21] (2024) Openrouter: A unified interface for LLMs. [Online]. Available: <https://openrouter.ai/>
- [22] A. Chen, C. Wong, B. Sharif, and A. Peruma, "Exploring code comprehension in scientific programming: Preliminary insights from research scientists," in *Proceedings of the 33rd IEEE/ACM International Conference on Program Comprehension (ICPC 2025)*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.10037>